

8-bit RISC Processor on FPGA

Diego Davalos
College of Engineering
California State University, Long Beach
Long Beach, USA
Diego.Davalos01@student.csulb.edu

Jonathan Lopez
College of Engineering
California State University, Long Beach
Long Beach, USA
Jonathan.Lopez03@student.csulb.edu

Matthew Kim
College of Engineering
California State University, Long Beach
Long Beach, USA
matthew.kim03@student.csulb.edu

Abstract—This project aims to develop a calculator using the 8-bit RISC processor architecture with Verilog and customized for specific implementation on the Nexys A7-100T FPGA board. Essential modules such as an ALU, Register Memory, and Control Unit will be programmed and interconnected via a Top module to form the processor's architecture. The FPGA implementation will feature user interaction via the board's slide switches and on-board push buttons, and will display results on 7-segment displays, demonstrating the processor's functionality as a calculator.

Additionally, this project explores solutions to hardware challenges like adders, multiplication, and division using the 8-bit RISC processor's capabilities. By addressing these challenges, this project seeks to provide efficient and scalable solutions within the constraints of an 8-bit architecture. Overall, this enterprise aims to deepen an understanding of computer architecture principles, FPGA design methodologies, and hardware-software interactions, culminating in a functional processor prototype with valuable insights into hardware design challenges.

Keywords—Addend, Sum, Subtrahend, Minuend, Difference, Dividend, Divisor, Quotient, Multiplier, Multiplicand, Product, Arithmetic Logic Unit, Reduced Instruction Set Computer

I. INTRODUCTION

Developing an 8-bit RISC processor effectively captures numerous salient concepts related to digital logic design that will be leveraged to simulate a digital calculator. The purpose of this simple calculator is to sufficiently express the functionality of our processor, by executing computations consisting of addition, subtraction, multiplication, and division. The (8) available 7-segment displays on our target technology (Nexys A7 trainer board) will serve as the primary interface to sufficiently express the results from computations, as well as concurrent inputs from the user, whereas the on-board buttons will be used to toggle through the available mathematical operations and for memory read and write. Furthermore, all (16) on-board switches will tie to the desired binary value that the user wishes to include within a selected arithmetic expression.

II. BACKGROUND & PRELIMINARIES

Since our hardware design hinges on the functionality of a digital 8-bit RISC processor, to this end, our initial research was centered on grasping the framework and interplay of a processor with our intended design. The computer architecture at hand—Reduced Instruction Set Computer (or RISC)—describes a particular model which is preceded by its former counterpart: Complex Instruction Set Computer (CISC) architecture. Given the nature of our ultimate goal, a RISC processor is favorable due to its affinity for a simple yet efficient approach towards instruction fetching and decoding. In addition, this option lends itself to amenable pipelining capabilities and better power consumption on the FPGA board. Most notably within our design, many relevant computations occur on values within simple instruction sets

and populated data registers sourced from user input: a feature emulating RISC architecture.

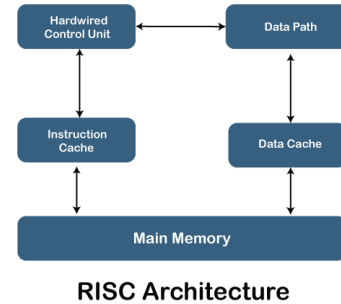


Fig. 1. RISC Architecture Diagram

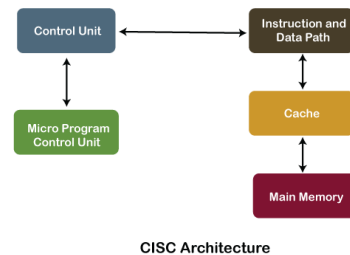


Fig. 2. CISC Architecture Diagram

As an extension to our original design, the processor has the ability to do basic math arithmetic such as addition, subtraction, multiplication, and some division. The processor also converts binary inputs from the user to decimal integers for streamlining calculations and convenience. All of the modules were designed in Verilog with special emphasis for structural design owed towards modules responsible for mathematical calculations. One significant caveat is that all calculations involve exclusively unsigned numbers.

Now, in order to procedurally execute addition and subtraction operations, we programmed an 8-bit Ripple-Carry Adder with subtraction capabilities by the select bit which determines whether we are adding or subtracting. For multiplication, our design encompasses multiple partial products and full adder trees that culminate into partial sums, similar to conventional multiplication with decimal numbers. On the other hand, our division implementation follows an inverted model of multiplication, consisting of subtractor trees and mimics bit shifting. For conversions from binary to decimal we developed a “filtering” module, *digit_Sep.v*, which essentially converts binary arithmetic results to decimal and channels them through a time multiplexer for display on the relevant 7-segment displays.

Ultimately, the images below showcase our final design with I/O considerations, and demonstrate the full functionality with the relevant connections:

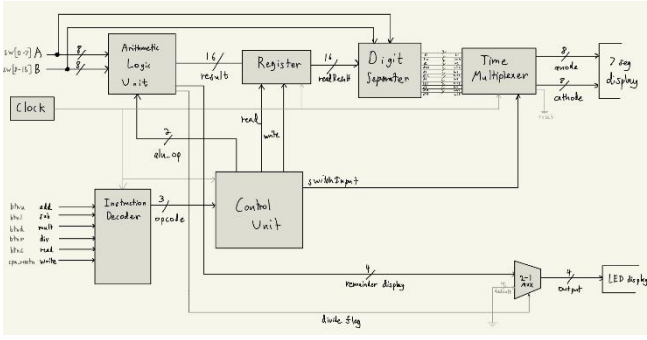


Fig. 3. Block Diagram of 8-bit Processor

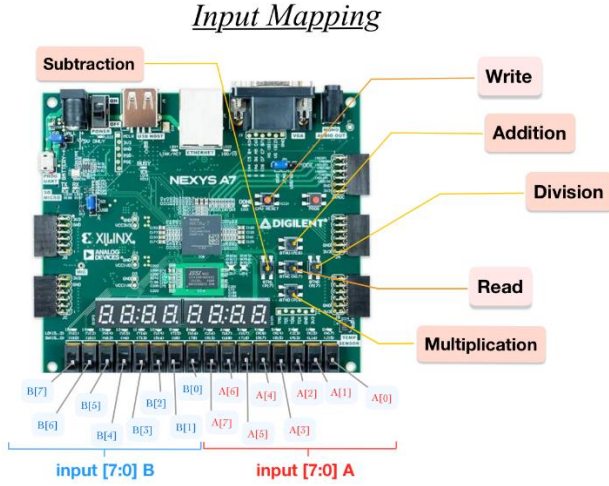


Fig. 4. Input Mapping on A7 Trainer Board

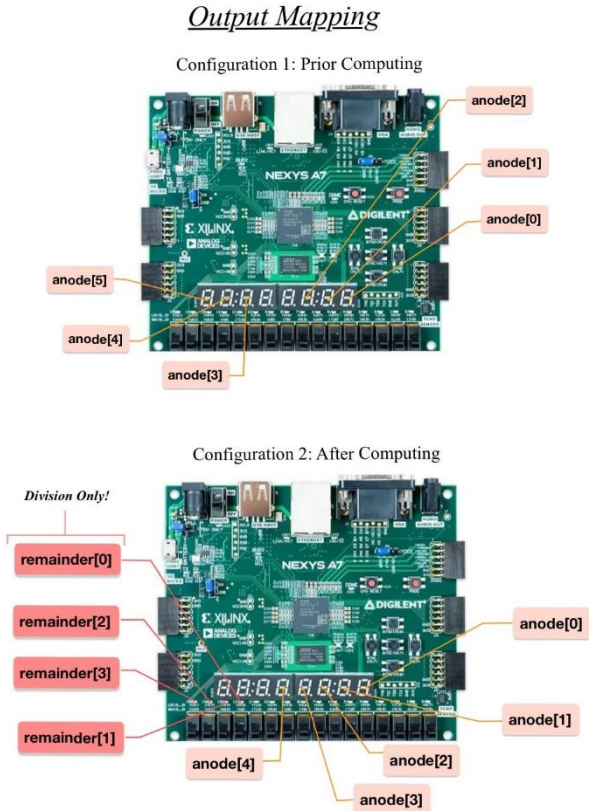


Fig. 5. Output Mapping on A7 Trainer Board (Before & After Computing)

Note: Output Mapping on the FPGA board evaluates with input A as the (left) addend, minuend, multiplicand, and dividend in each corresponding arithmetic computation. Therefore, input B represents the (right) addend, subtrahend, multiplier, and divisor, respectively.

III. IMPLEMENTATION

A. Instruction Decoder (*InstructionDecoder.v*)

In order to incorporate the onboard button inputs from the FPGA board, this module is synchronized via a clock signal and consists of a series of six 2-to-1 muxes with each separate input (addition, subtraction, multiplication, division, write, and read) acting as the select signal. The input of each mux consists of the default opcode value if select signal is low, and the opcode value desired for the respective input select signal. The main purpose of the instruction decoder is to take in various signals and decode to its respective output of one 3-bit register called “opcode”, short for “operation code”. This operation code is delivered to the Control Unit. This module is necessitated because of the multiple input signal nature of button presses and the desired organization of one input signal entering the Control Unit, thus keeping the processor scalable and generalized.

Input Instruction Signals	Decoded Operation Code		
Add 1	0	0	0
Subtract 1	0	0	1
Multiply 1	0	1	0
Divide 1	0	1	1
Read 1	1	0	0
Write 0	1	0	1
Default 0	1	1	0

Table 1. Instruction Decoder I/O Table

B. Control Unit (*ControlUnit.v*)

This module simulates the control unit of the CPU, responsible for decoding the opcode and generating the necessary control signals to execute the corresponding desired instruction. It determines how data flows through the CPU and controls the various functional units (such as the ALU and register file) based on the operation specified by the opcode. It is also synchronized via a global clock signal.

Input					Output			
Opcode	ALU op	R	W	Switch Input	Operations Performed			
0	0	0	0		0	0	0	Add
0	0	1	0		1	0	0	Sub
0	1	0	1		0	0	0	Multiply
0	1	1	1		1	0	0	Divide
1	0	0	x		x	1	0	Read
1	0	1	x		x	0	1	Write
Default	x	0	0	1	Display User Inputs			

Table 2. Instruction Decoder I/O Table

C. Arithmetic Logic Unit (ALU.v)

This project design encompasses an Arithmetic Logic Unit (ALU) that is capable of addition, subtraction, array multiplication and limited division operations. It instantiates a Ripple-Carry-Adder and Subtractor as well as an Array Multiplier and Divider module to perform operations with the two 8-bit inputs (A and B) based on the provided opcode. The ALU receives the control signals sent by the Control Unit (a 2-bit ALU opcode with each of its four states assigned to an operation), and is used as the select signals for a 4-to-1 mux that outputs the desired 16-bit result. As an aside, each operation calculates the result every positive edge of a clock signal which subtly interferes with efficiency, introducing an optimization issue caused by the simultaneous instantiation of all four arithmetic operation modules.

D. Memory Register (Register.v)

This module involves implementation of a register file, which is a fundamental component in digital systems for storing data during computation or processing. Due to the limitations of having only one button for read and another for write, we opted to only include one 16-bit register for storing the desired result in memory (16 JK Flip-Flops). If the write signal is enabled, the current result is stored in the memory register. A 2-to-1 multiplexer takes in the result from memory as well as the current result. If the read signal is enabled, the result from memory is sent to the output (towards the Digit Separator Module). This module's flip-flops are synchronized via a clock signal as well.

E. Digit Separator (digit_Sep.v)

This module handles converting the two 8-bit inputs as well as the 16-bit result into decimal digits (ones, tens, hundreds, and thousands place) for input on the 7-segment display. In particular, this module will autonomously convert binary expressions from the ALU, given the user's desired inputs for a general calculation or the specific calculation in mind, into decimal form.

F. Time-Multiplexing (time_multiplexer.v)

A feature for providing multiple signals to each individual 7-segment display is viable through time-multiplexing; namely, we employ a signal connection scheme whereby each relevant display is perceptibly distinct to the human eye. For our purposes, this technique enables multiple signal transmission when the user performs a calculation, via (5) 4-bit inputs which tie to their respective cathode display orientation, as described in *BCD to 7Seg.v*, over a shared channel with allocated time corresponding to each incoming signal in a fixed timeframe. The Nexys A7-100T has (8) 7-segment displays that, by design, share a common anode that must be driven low to illuminate that specific display. Further, each display consists of (7) separate LED segments with (1) decimal point whose cathode terminal with a logic '0' illuminates them.

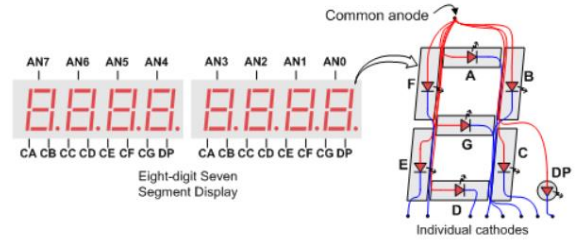


Fig. 6. 7-segment Anode and Cathode Configuration

For distinct characters like decimal numbers to properly display on the five rightmost 7-segment displays, we provide 4-bit input signals a suitable refresh rate, which is ideally around 1 kHz, since at this rate the darkening of a digit cannot be perceived by the human eye before it is illuminated again.

We utilize equation (1) to reliably determine an array width for an n-bit binary up counter used in our time-division multiplexer:

$$\text{Refresh Rate} = \frac{\text{System Clock}}{2^n} \quad (1)$$

For our project, an ideal width was $n = 17$ which gives a $\text{Refresh Rate} \approx 763 \text{ Hz}$ — a value that approximately equates to 1.31 ms. Therefore, a suitable refresh rate for each required 4-bit input signal, for our needs, should be approximately 1.31 ms for a total duration of 6.55 ms in which the (5) rightmost 7-segment displays have fully traversed their allotted time.

As for our binary up counter, we employ two instances: one for tracking the current state that shall be incremented by another instance entitled *next_state*. We isolate the three most significant bits of the binary counter *curr_state* to cycle between the available options in the time-divider multiplexer. Each cycle lasts about 763 Hz or 1.31 ms: the duration for an individual 4-bit input that corresponds to one of the 7-segment displays. Therefore, as indicated in *time_multiplexer.v*, each 7-segment display has its respective 4-bit input given by *in1* tied to the ones place, *in2* corresponding to the tenths, *in3* with the hundredths place, and so on. We also utilized another set of similar inputs titled *in1_A*, *in2_A*, *in3_A*, *in4_B*, etc. These inputs, on the other hand, are tailored towards configuring the 8-bit calculator in a practical manner, by displaying the relevant decimal numbers the user wishes to include prior to computations. When the user presses a push button to calculate, a select signal originating from the Control Unit determines the appropriate time-multiplexing scheme.

G. 8-bit Ripple Carry Adder & Subtractor (AddSub_8bit.v)

An 8-bit Ripple Carry Adder and Subtractor serves as the primary algorithm for adding and subtracting. It receives two 8-bit inputs and produces one 8-bit output. The functional basis for this 8-bit RCAnS stems from eight individual cascading full adders that depend on contiguous full adders to produce meaningful computations. Notably, the full adder (FA) is a logical unit that receives two 1-bit sized inputs and a carry-in signal (C_{in}) that essentially adds these inputs

together. The resulting outputs named *sum* is 1-bit as well as the carry-out (C_{out}).

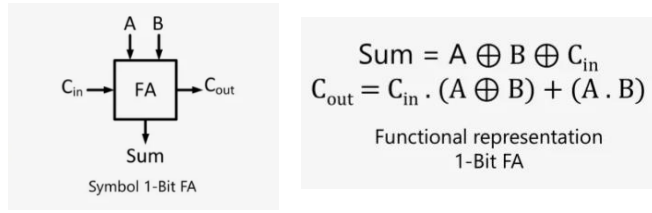


Fig. 7. One-bit Full Adder Diagram and Logical Representation

Now, with respect to our 8-bit RCAnS, the rippling result from each preceding FA must channel its C_{out} to the succeeding FA with the least significant bit in this chain as the inception point.

For subtraction, we can utilize the carry-in signal from the least significant bit as a selection line to enable this feature; however, we must also embed an exclusive-or gate (XOR) to each FA to emulate two's complement. Now one of the inputs to the FA must be driven into the XOR gate along with the selection line from the C_{in} signal originating from least significant bit. The diagram below demonstrates the nature of our 8-bit RCAnS implementation:

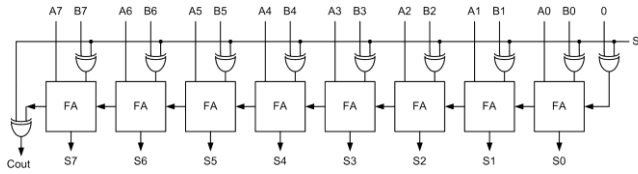


Fig. 8. 8-bit Ripple Carry Adder and Subtractor

H. 8-bit Array Multiplier (*ArrMult_8bit.v*)

In this implementation we designed an 8-bit Array Multiplier by creating a network of partial products and a full adder tree that funnel into partial sums. This multiplier considers two 8-bit inputs and produces one 16-bit result. For this featured design, significant inspiration was drawn from a 4-bit Array Multiplier studied during laboratory one in our computer engineering course: Digital Design Techniques and Verification. We specifically employ a similar design process, given by *Figure 9* below, by extending a 4-bit Array Multiplier algorithm to consider 8-bit inputs.

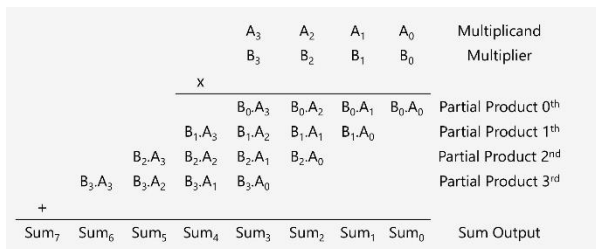


Fig. 9. 4-bit Array Multiplier Algorithm

The partial products are modeled from numerous 2-to-1 multiplexers that generate the necessary partial products.

Then, we utilize an extended full adder tree to produce the partial sums which cascade as the *sum* output.

The full adder tree also contains several instances of full adders and half adders (HA) which enables addition of the relevant partial products along each respective column. For the half adders, this logic unit is akin to full adders except for the lack of a carry-in signal. That is, half adders accept two 1-bit sized inputs and provide two 1-bit outputs: a *sum* and carry-out (C_{out}) output. The diagram below demonstrates a half adder for additional clarity:

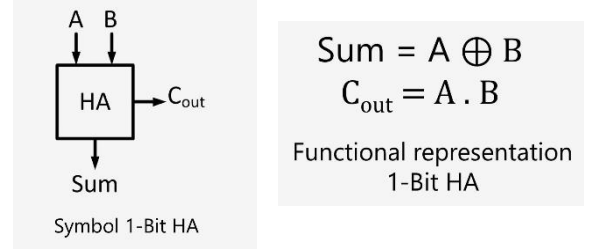


Fig. 10. One-bit Half Adder Diagram and Logical Representation

Ultimately through our design process, we fully extend the full adder tree by extrapolating additional connections between adjacent half adders and full adders to extend the functionality of a conventional 4-bit array multiplier and consider 8-bit inputs, virtually doubling its input and output capacity. Here, the next image given below has many parallels with our design except that it features AND gates for evaluating the partial products. Our model, on the other hand, uses 2-to-1 multiplexers for evaluating these variables.

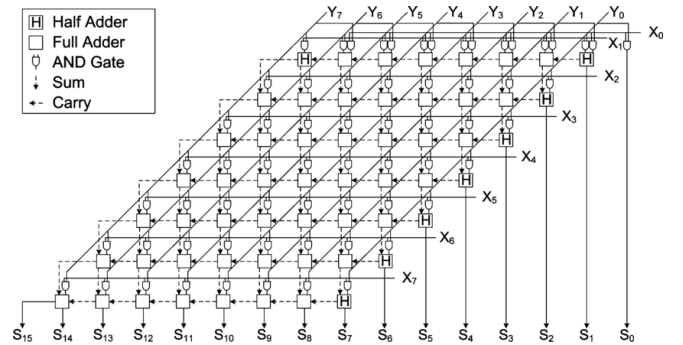


Fig. 11. Diagrammatic 8-bit Array Multiplier Using AND Gates

I. 4-bit Divider (*divider_4bit.v*)

The divisor algorithm characterized by our dividing machine utilizes numerous subtractors along with cascading 2-to-1 multiplexers to channel relevant calculations for the quotient and remainder. Notably, our 4-bit Divider is an inversion of the algorithm demonstrated by the array multiplier. Our design is modeled by *Figure 12* with each row representing four iterations of subtraction from the most significant bit to the least significant bit of the dividend by the divisor.

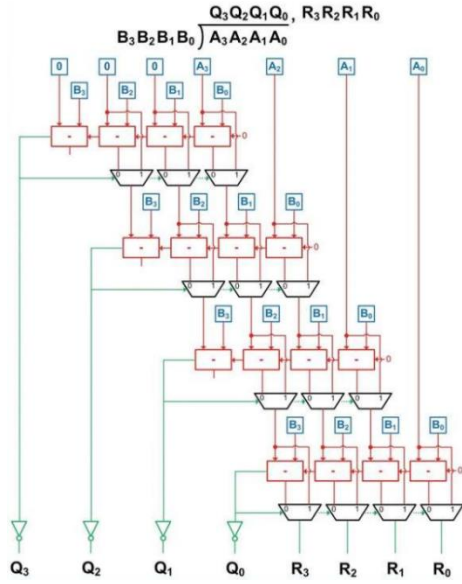


Fig. 12. Theoretical Model of 4-bit Divider (Iterative Subtractor Tree)

The multiplexers provide a restorative feature after each computation of subtraction depending on the selection bit given from the calculation prior. The negation of the selection bit will provide the quotient starting from the most significant bit to the least from each successive row of subtraction. Theoretically, if the sign bit given from each subtraction produces a negative, the next iteration considered for subtraction should be “reset” or restored because the calculation was invalidated. In other words, the next iteration for subtraction will involve a subtrahend that is not initially negative given by its sign bit. Emphatically, without a flag to determine if a subtraction at each rung is permissible, the resulting quotient and remainder is therefore meaningless because it deviates from the conventional principles to simulate long division.

Provided that two 4-bit inputs, the dividend and divisor, are applied to this algorithm two 4-bit outputs generally known as the quotient and remainder will be resulted, assuming that the divisor is **not** zero. The ultimate result from this iterative subtraction yields a 4-bit expression of the remainder in full.

J. Top Module (*tempTop.v*)

A holistic assessment for verifying the functionality of our 8-bit RISC processor is given by the Top module which unifies all relevant instances to simulate our final project design. In particular it instantiates all relevant modules that encompass our 8-bit processor, such as the Instruction Decoder, Control Unit, ALU, and registers. Other additional modules which enable seamless interfacing from conversions to the 7-segment displays is supported by the synergy between *digitSep.v* and *time_multiplexer.v*.

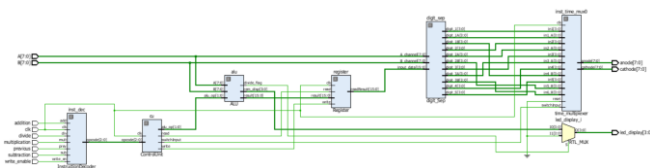


Fig. 13. Top Module of 8-bit Risc Processor

IV. EVALUATION

Our finalized project was nearly realized in all aspects of its initial design. Some of the design challenges were simply too difficult to include, such as conversions from a binary representation to hexadecimal, for example, which was owed primarily to the limitations of the 7-segment display. In other areas, we wanted to expand memory allocation to save many user calculations but was ultimately redundant when we exhausted all push-buttons and slide switches for our most basic user interface. Though we wished to consider other additional features to develop a more sophisticated and versatile computing instrument, timing constraints ultimately precluded further arrangements for implementation. The dividing algorithm, for example, employed in our processor is **not** functional for general purposes. We considered a simpler design—a 4-bit Divider—before isolating a more challenging design like an 8-bit Divider. In this architecture, we produce correct results for very specific cases but provide nonsensical results for all other cases. In retrospect, we assume that this stems from improper implementation of the iterative subtractor algorithm; though, it is also entirely possible that the algorithm is wrong altogether. If we were to properly simulate long division in Verilog, we also would have elevated our design to include 8-bit inputs.

That said, given that our divider is the most prominent shortcoming of our project, we wish to describe the verification process tied to this design and provide it full transparency. In addition, given the breadth of calculations possible, we examined all arithmetically-inclined modules with exhaustive self-checking testbenches. For the 4-bit divider, we were able to isolate all instances by which it fails to produce the correct quotient and remainder. In some cases, the quotient was correct, but the remainder was wrong. Conversely, other cases involved the proper remainder, but the quotient was incorrect. Due to timing constraints, we were unable to fully develop a completely functional divider as indicated by the testbench in *Figure 13 (A)* and *Figure 14 (B)*.



Fig. 14. (A) Exhaustive Self-checking Testbench Waveform of 4-bit Divider

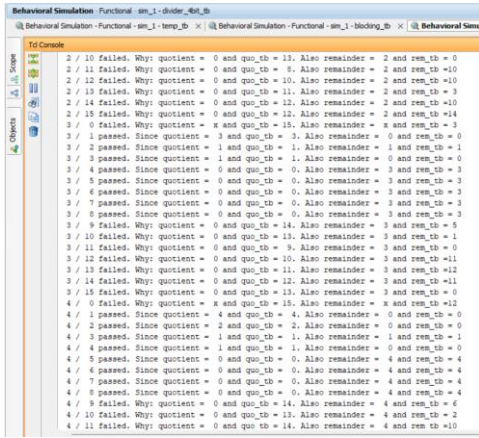


Fig. 15. (B) (Subsection) Exhaustive Self-checking Testbench Console of 4-bit Divider

Now for other testbenches, like the 8-bit Ripple-Carry Adder and Subtractor, we demonstrate a design with complete functionality. Given additional time, we likely would have considered a more elegant algorithm, such as an 8-bit Carry-Look-Ahead Adder and Subtractor or an 8-bit Carry-Save Adder with subtraction capabilities. Nonetheless, our 8-bit RCAnS exhibited by the next couple figures showcase accurate results with any given permutation for calculating addition and subtraction with unsigned numbers.

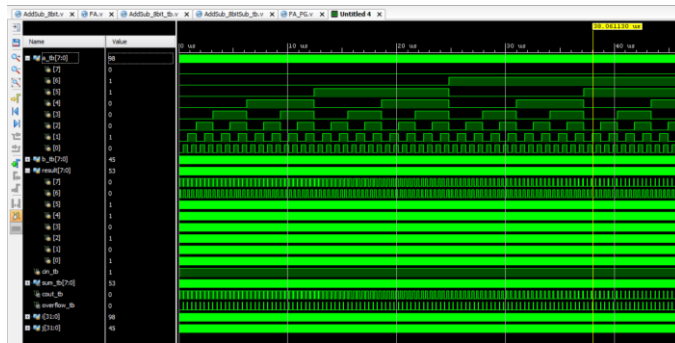


Fig. 16. (A) Exhaustive Self-checking Testbench Waveform of 8-bit RCAnS (Subtraction)

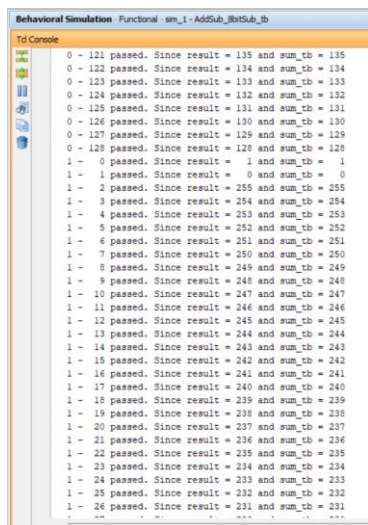


Fig. 17. (B) (Subsection) Exhaustive Self-checking Testbench Console of 8-bit RCAnS (Subtraction)

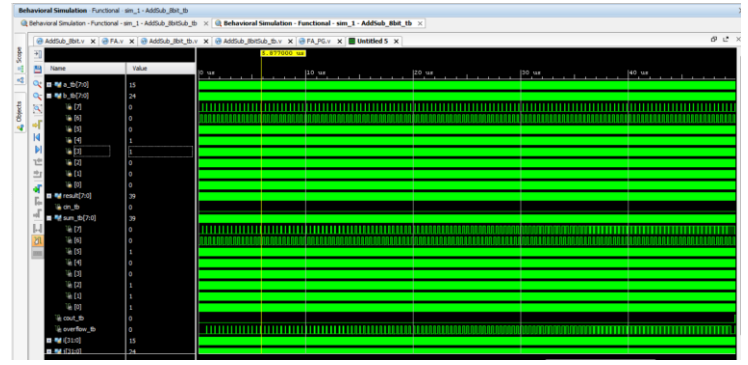


Fig. 18. (C) Exhaustive Self-checking Testbench Waveform of 8-bit RCAnS (Addition)

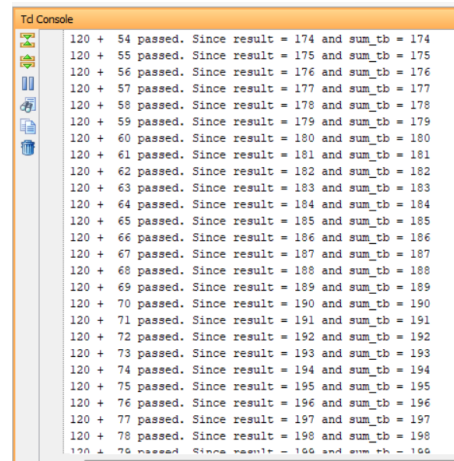


Fig. 19. (D) (Subsection) Exhaustive Self-checking Testbench Console of 8-bit RCAnS (Addition)

In addition to the implementation above, another successful module that computes results with full confidence is the 8-bit Array Multiplier. This is demonstrable by the waveform in *Figure 19 (A)* and console output in *Figure 20 (B)* which relates to our verification method for ascertaining its integrity.

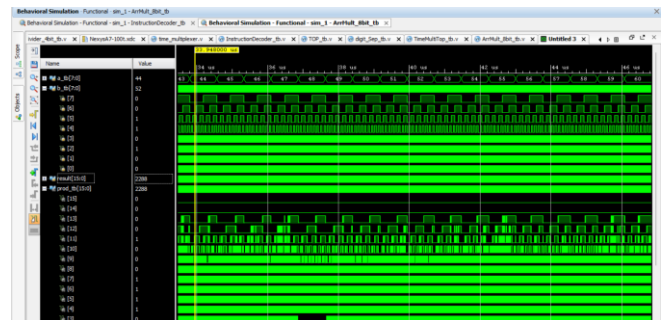


Fig. 20. (A) Exhaustive Self-checking Testbench Waveform of 8-bit Array Multiplier

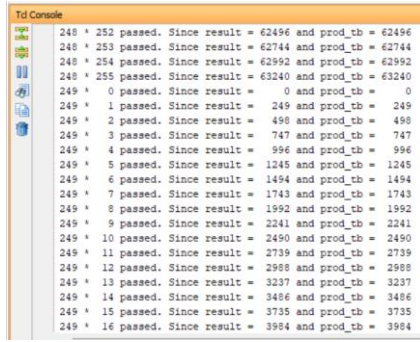


Fig. 21. (Subsection) Exhaustive Self-checking Testbench Console of 8-bit Array Multiplier

The time-multiplexing scheme we used, for displaying multiple distinct decimal numbers on the 7-segment displays, were verified through a testbench carefully demonstrating the precise moments each anode receives a signal to illuminate a specific decimal number. Though each signal is briefly illuminated, the testbench illustrates the behavior of time-division multiplexing and its intended functionality for on-board purposes. For example, at time 12.48 ms into the simulation, we can observe the case statement in the 20-bit up binary counter *curr_state* where the rightmost display has sole access to the 7-segment display channel. This is indicated by the result from *anode_tb* showing 11110 and thus decimal (2) on the cathodes. At the next increment along *curr_state*[17], the next number decimal (5) in queue, is provided the signal to display on the 7-segment display. This dynamic occurs in brief and seamless intervals, such that the flickering of the 7-segment displays is unnoticeable to the human eye.

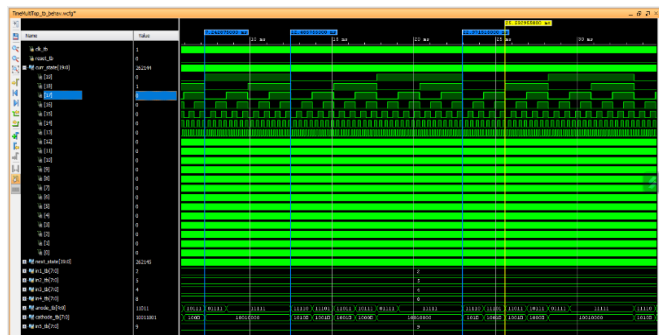


Fig. 22. (A) Time-divison Multiplexing Testbench (Zoomed Out)

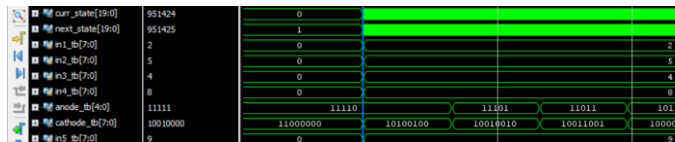


Fig. 23. (B) Time-divison Multiplexing Testbench (Magnified)

The ultimate result from our functional time multiplexer is demonstrated by the distinct decimal numbers relayed to the user interface on the FPGA board. Given inputs from the user, the displays will provide relevant information considered for specific or general calculations.

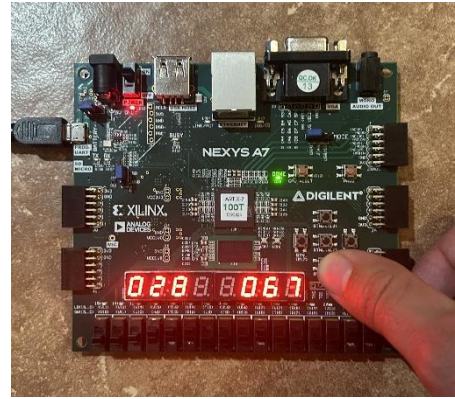


Fig. 24. (B) Running 8-bit RISC Processor (Displaying User Inputs)

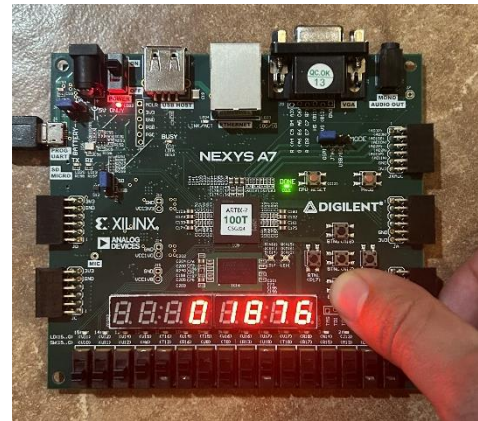


Fig. 25. (B) Running 8-bit RISC Processor (Calculating Multiplication)

V. CONCLUSION

The implementation focus of this project was centered on designing a specialized 8-bit RISC processor with adaptations for performing real-time computations through a simplified user interface. Hardware design challenges concerning the arithmetic modules proved to be particularly insightful for structural design, whereas the RISC architecture provided an organized framework for streamlining instruction sets. Verifying that our results would be consistent via simple and exhaustive self-checking testbenches lended additional confidence that our intended implementations would be accurate and consistent. Within the shortcomings of our design, we were able isolate specific design flaws and provide the necessary corrections therein. On one hand, though timing constraints ultimately precluded further arragments for refining our dividing algorithm, we were successful in utilizing verification principles to uncover some of the erroneous aspects of this design. Furthermore, we exhausted most options for on-board interfacing by equipping our design with several features to showcase its versatility as a computing instrument. In closing, this project provided valuable insights into the intricacies of hardware design, particularly in the context of developing specialized processors. Through meticulous design iterations, rigorous testing, and thoughtful consideration of user requirements, we have created a relatively robust computing platform capable of meeting real-time computational demands for practical settings with the Nexys A7-100T FPGA board.

REFERENCES

- [1] J. Jeemon, "Pipelined 8-bit RISC processor design using Verilog HDL on FPGA," *IEEE Conference Publication | IEEE Xplore*, May 01, 2016. <https://ieeexplore.ieee.org/document/7808194> [accessed Apr. 07, 2024].
- [2] "Difference Between RISC and CISC - javatpoint," www.javatpoint.com. <https://www.javatpoint.com/risc-vs-cisc> [accessed Apr. 13, 2024].
- [3] S. SAD and S. PAWAR, "Design and Implementation of 16 Bit RISC Processor in VHDL Using Xilinx Tools," 4, Apr. 2018. Accessed: Apr. 21, 2024. [Online]. Available: <https://jier.co.in/download/8.Sheetesh%20pp%2028-31.pdf>
- [4] S. P. Dandamudi, *Guide to RISC Processors*. Springer Publishing Company, 2004. [Online]. Available: <https://library.lol/main/4DEF730EAE6F22F4A491C972DA2B13C5>
- [5] "RISC vs. CISC," *Risc Architecture*. <https://cs.stanford.edu/people/eroberts/courses/soco/projects/risc/riscisc/> [accessed Apr. 21, 2024].
- [6] D. M. Harris and S. L. Harris, *Digital design and computer architecture*. Morgan Kaufmann, 2015. [Online]. Available: <https://allbooksfordownloading.wordpress.com/wp-content/uploads/2017/01/digital-design-and-computer-architecture-by-david-and-sarah-harris.pdf>
- [7] A. Brown, "NeXyS A7 Reference Manual - Digilent Reference," Apr. 27, 2023. <https://digilent.com/reference/programmable-logic/nexys-a7/reference-manual> [accessed May 01, 2024].
- [8] "Digital circuits - counters," *Tutorialspoint*. https://www.tutorialspoint.com/digital_circuits/digital_circuits_counters.htm [accessed May 01, 2024].
- [9] P. P. Chu and Cleveland State University, *FPGA PROTOTYPING BY VERILOG EXAMPLES*. WILEY, 2008.
- [10] "Digital circuits - counters," *Tutorialspoint*. https://www.tutorialspoint.com/digital_circuits/digital_circuits_counters.htm [accessed May 01, 2024].
- [11] GeeksforGeeks, "Computer Organization and Architecture Pipelining Set 1 (Execution, Stages and Throughput)," *GeeksforGeeks*, Aug. 09, 2023. <https://www.geeksforgeeks.org/computer-organization-and-architecture-pipelining-set-1-execution-stages-and-throughput/#> [accessed May 01, 2024].
- [12] B. J. LaMeres, *Introduction to Logic Circuits & Logic Design with Verilog*. Springer, 2019. [Online]. Available: <https://electrovolt.ir/wp-content/uploads/2018/03/Logic-Circuits-Logic-Design-with-Verilog-ElectroVolt.ir .pdf>
- [13] GeeksforGeeks, "Frequency Division and Time division multiplexing," *GeeksforGeeks*, Mar. 25, 2023. <https://www.geeksforgeeks.org/frequency-division-and-time-division-multiplexing/> [accessed May 01, 2024].